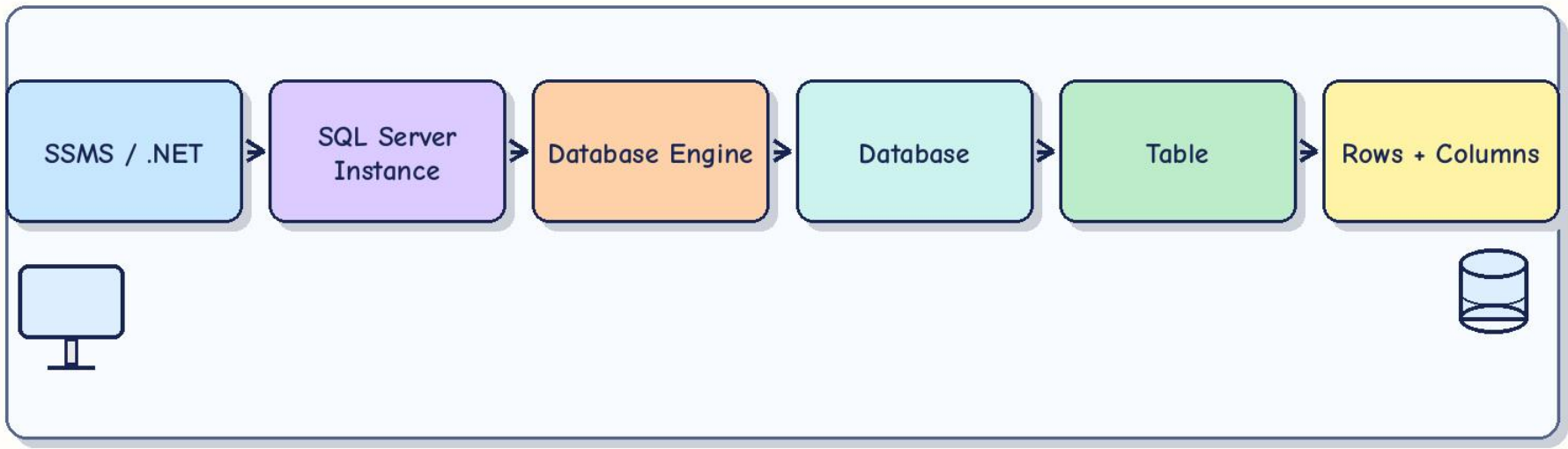
REVISION CHEAT SHEET:

SQL SERVER FUNDAMENTALS

Interview-ready notes • examples • SQL • diagrams

SQL Server Fundamentals



These concepts are closely connected, so first see the complete picture:

```text
User / Application
    ↓
Client
SSMS / .NET Application
    ↓
Connection
    ↓
SQL Server Instance
    ↓
Database Engine
    ↓
Database
    ↓
Table
    ↓
Rows + Columns
```

For example:

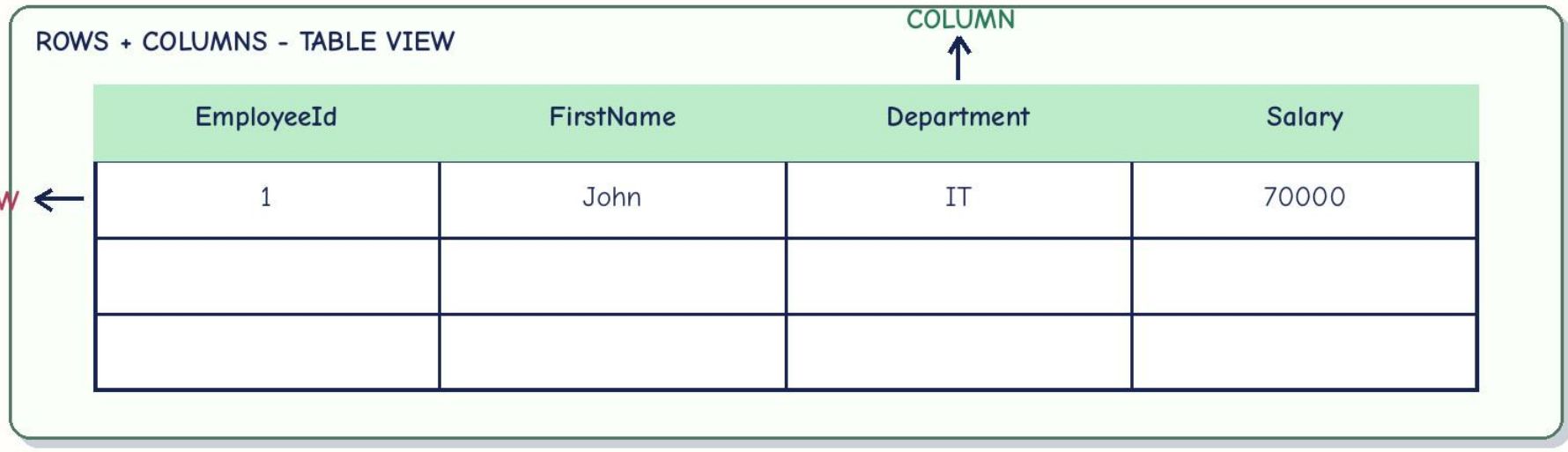
```text
SSMS
↓
MYPC\SQLDEV
↓
SQL Server Database Engine
↓
EmployeeDb
↓
```

TEXT (CONT.)

```
Employees
↓
Rows and Columns
```

Understanding this flow makes all the individual concepts much easier.

Rows and Columns



SQL Server stores relational data mainly inside tables.

A table organizes data into:

TEXT

```
Rows
+
Columns
```

Microsoft describes table data as logically organized into rows and columns. A row represents a record, while a column represents an attribute or field of that record.

Consider this Employees table:

EmployeeId	FirstName	Department	Salary
1	John	IT	70000
2	Priya	HR	65000
3	David	Finance	75000

What is a Column?

A column represents one particular property or type of information.

Here:

TEXT

```
EmployeeId
FirstName
Department
Salary
```

TEXT (CONT.)

are columns.

Think:

Column = What information are we storing?

For example:

```
TEXT
FirstName
```

means this column stores an employee's first name.

```
TEXT
Salary
```

means this column stores salary information.

What is a Row?

A row represents one complete record.

For example:

```
TEXT
1 | John | IT | 70000
```

is one row.

It represents one employee.

Another row:

```
TEXT
2 | Priya | HR | 65000
```

represents another employee.

Think:

Row = One complete item or record.

Easy Way to Remember

```
TEXT
Column
=
One property
```

TEXT (CONT.)

```
Row
=
One complete record
```

Example:

TEXT

```
Employees

EmployeeId | Name  | Department
---------------------------------
1          | John  | IT
2          | Priya | HR
```

Here:

TEXT

```
Name
```

is a column.

And:

TEXT

```
1 | John | IT
```

is a row.

Creating Rows and Columns in SQL Server

When we create a table, we define its columns.

SQL

```sql
CREATE TABLE Employees
(
  EmployeeId INT PRIMARY KEY,
  FirstName VARCHAR(50) NOT NULL,
  Department VARCHAR(50) NOT NULL,
  Salary DECIMAL(10,2) NOT NULL
);
```

Here:

SQL

```sql
EmployeeId INT
```

creates a column named EmployeeId.

```sql
FirstName VARCHAR(50)
```

creates another column.

The data type tells SQL Server what kind of values the column can store.

Now:

```sql
INSERT INTO Employees
(
  EmployeeId,
  FirstName,
  Department,
  Salary
)
VALUES
(
  1,
  'John',
  'IT',
  70000
);
```

creates a row.

So:

```text
CREATE TABLE
  ↓
Defines columns

INSERT
  ↓
Creates rows
```

Selecting Columns

If you write:

```sql
SELECT FirstName, Salary
FROM Employees;
```

you are asking SQL Server for two columns.

Result:

```
FirstName   Salary
------------------
John        70000
Priya       65000
David       75000
```

Selecting Rows

If you write:

```sql
SELECT *
FROM Employees
WHERE EmployeeId = 2;
```

SQL Server finds the row where:

```
EmployeeId = 2
```

Result:

```
EmployeeId   FirstName   Department   Salary
-----------------------------------------------
2            Priya       HR           65000
```

Row vs Column

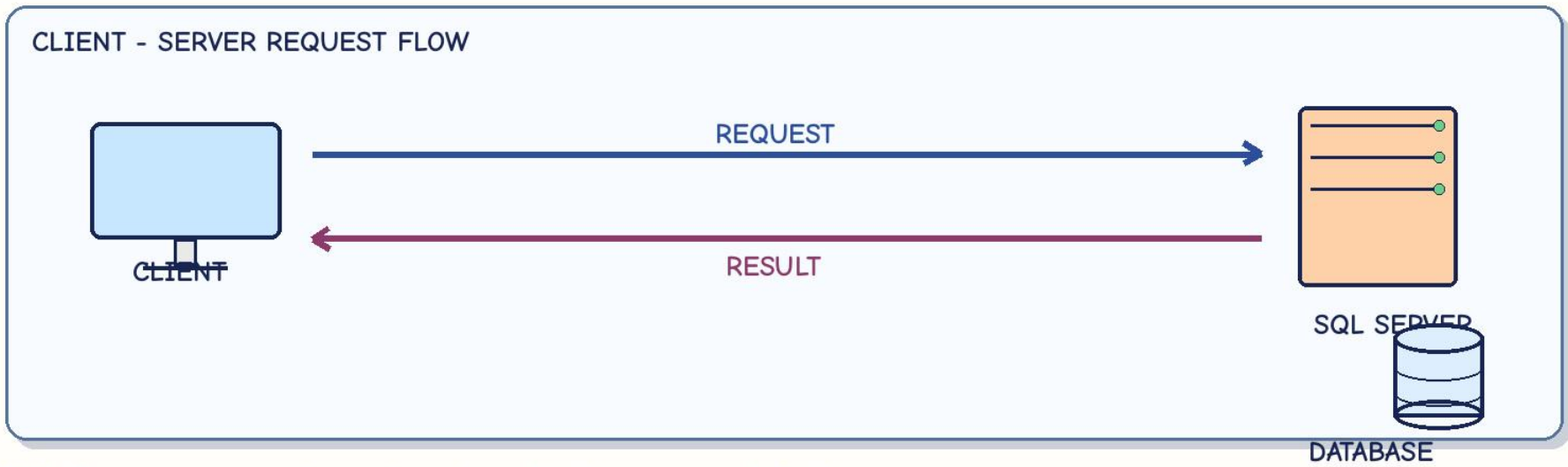
Row	Column
One complete record	One property of records
Horizontal	Vertical
Example: one employee	Example: employee salary
Contains values for several columns	Contains values of the same type/purpose across rows
Added commonly using INSERT	Defined when designing/altering the table

Remember:

```
Column = What?

Row = Which record?
```

Client-Server Database Architecture



SQL Server commonly works using client-server architecture.

There are two important sides:

```text
Client
+
Server
```

Client

The client is the application or tool requesting something from SQL Server.

Examples:

```text
SSMS
ASP.NET Core Web API
Desktop application
PowerShell script
Reporting application
```

The client sends requests to SQL Server.

For example:

```sql
SELECT *
FROM Employees;
```

Server

The server side contains the SQL Server Database Engine that processes the client's request and accesses databases.

Conceptually:

```text
Client
 ↓
Request
 ↓
SQL Server
 ↓
Database
 ↓
Result
 ↓
Client
```

Simple Client-Server Example

Suppose you open SSMS and execute:

```sql
SELECT FirstName
FROM Employees
WHERE EmployeeId = 1;
```

The process is approximately:

```text
SSMS
↓
Sends SQL request
↓
SQL Server instance
↓
Database Engine processes request
↓
Reads Employees table
↓
Finds EmployeeId = 1
↓
Returns result
↓
SSMS displays John
```

Result:

```text
FirstName
---------
John
```

Client and Server Can Be on the Same Computer

During learning, you may have everything installed on your laptop:

```text
Your Computer
│
├── SSMS
│     ↓
│
└── SQL Server
```

SSMS is still logically the client, while SQL Server is providing the database service.

Client and Server Can Be on Different Computers

In a real application:

```text
Developer Computer
   ↓
  SSMS
   ↓
Network
   ↓
Database Server
   ↓
SQL Server
```

Or:

```text
Web API Server
   ↓
Network
   ↓
SQL Server Database Server
```

SQL Server supports connections where the application and Database Engine are on separate machines; when they are separate, requests travel over a network connection.

Why Use Client-Server Architecture?

It allows one central SQL Server system to serve many applications or users.

For example:

```text
SSMS ──────────┐
               │
```

TEXT (CONT.)

```
.NET API ──────┼──→ SQL Server
         |
Reporting App ──┘
```

Instead of every application maintaining its own independent copy of business data, they can connect to a centrally managed database system.

Benefits include:

- Centralized data
- Controlled access
- Multiple clients
- Easier administration
- Better security management
- Shared business information

How Does the Client Connect?

A client generally needs information such as:

TEXT

```
Server / Instance name
Authentication information
Database
Network connection
```

For example:

TEXT

```
Server:

MYPC\SQLDEV
```

or for a default instance:

TEXT

```
MYPC
```

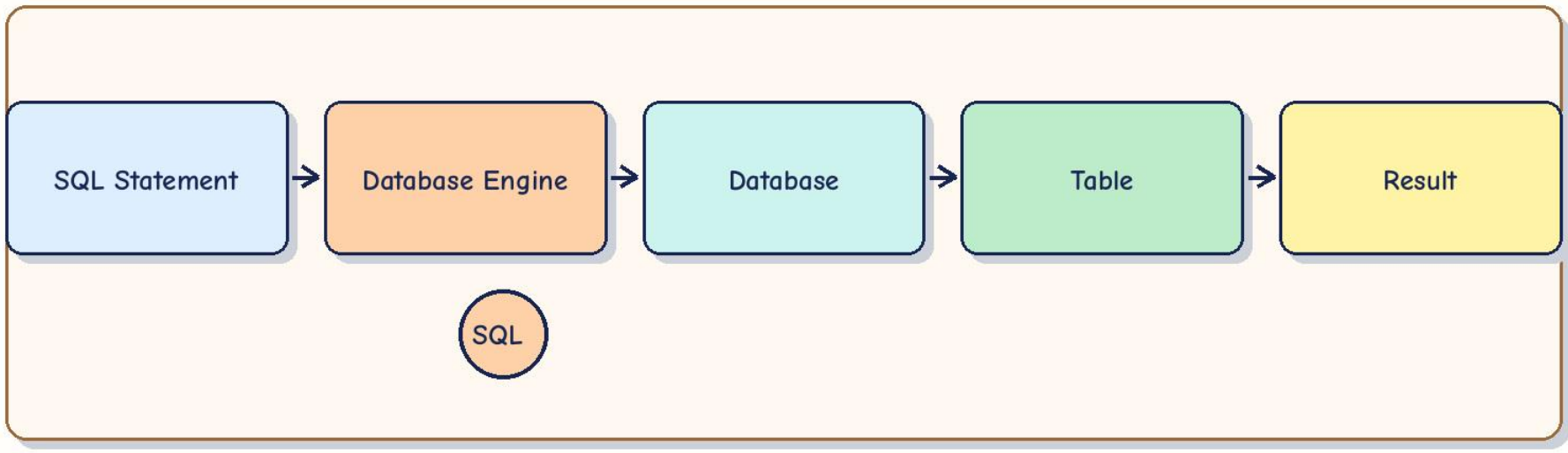
The default TCP port normally associated with a default SQL Server instance is 1433, although administrators can configure different ports. SQL Server can use protocols including TCP/IP, Named Pipes, and Shared Memory depending on the environment.

You do not need to memorize networking details yet.

The important concept is:

```text
Client
 ↓
Connection
 ↓
SQL Server Instance
```

Database Engine



The SQL Server Database Engine is the core component responsible for actually working with your relational database data.

Microsoft describes the Database Engine as the core service responsible for storing, processing, and securing data.

Simple definition:

> **The Database Engine is the core part of SQL Server that processes database requests and manages the data.**

Suppose you send:

```sql
SELECT *
FROM Employees;
```

The Database Engine handles the request.

Conceptually:

```text
SQL statement
  ↓
Database Engine
  ↓
Database
  ↓
Table
  ↓
Result
```

TEXT (CONT.)

What Does the Database Engine Do?

The Database Engine performs important database work such as:

TEXT

```
Receiving SQL commands

Finding requested data

Inserting data

Updating data

Deleting data

Processing queries

Managing transactions

Checking permissions

Maintaining data consistency

Returning results
```

For example:

SQL

```
UPDATE Employees
SET Salary = 75000
WHERE EmployeeId = 1;
```

The Database Engine processes the statement and performs the required database operation.

Database Engine Example

Suppose the table contains:

TEXT

```
EmployeeId   Name    Salary
---------------------------
1            John    70000
```

You send:

```sql
UPDATE Employees
SET Salary = 75000
WHERE EmployeeId = 1;
```

Conceptually:

```text
Client sends UPDATE
    ↓
Database Engine receives it
    ↓
Checks statement
    ↓
Finds EmployeeId 1
    ↓
Updates Salary
    ↓
Returns result/status
```

Afterward:

```text
EmployeeId   Name    Salary
----------------------------
1            John    75000
```

SQL Server vs Database Engine

These terms are related but should not be confused.

SQL Server refers to Microsoft's overall database platform/product.

The Database Engine is the core service within SQL Server that performs relational database processing.

For your current learning, think:

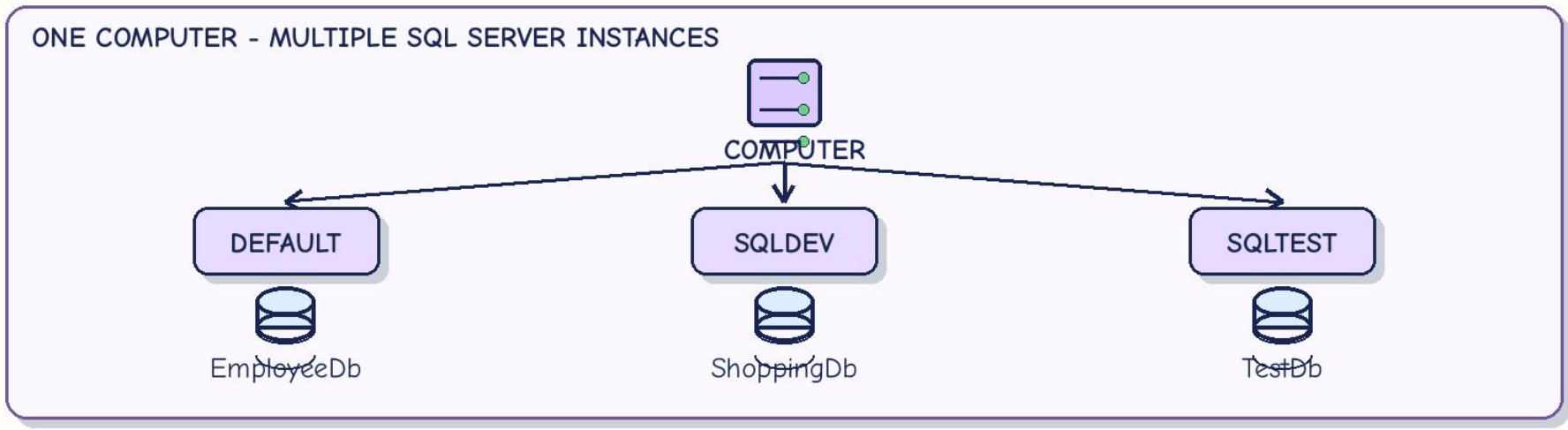
```text
Microsoft SQL Server
    ↓
Database Engine
    ↓
Databases
```

You will often hear developers casually say:

```text
"SQL Server processed the query."
```

More technically, the Database Engine is doing the core database-processing work.

SQL Server Instance



This is one of the most important concepts to understand correctly.

An instance is a running installation/copy of the SQL Server Database Engine that operates independently and manages its own databases.

Microsoft describes an instance of the Database Engine as a running copy of sqlservr.exe operating as an operating-system service. Each instance manages system databases and one or more user databases.

Simple definition:

> **A SQL Server instance is an independent running SQL Server Database Engine environment.**

Think:

```text
Computer
 ↓
SQL Server Instance
 ↓
Databases
```

For example:

```text
MyComputer
  ↓
SQLDEV
  ↓
EmployeeDb
ShoppingDb
FundraisingDb
```

Why Do We Need an Instance?

The client needs to know which running SQL Server environment it should connect to.

Suppose one computer has:

```text
SQLDEV
SQLTEST
```

These can be separate SQL Server instances.

```text
Computer
│
├── SQLDEV
│     ├── EmployeeDb
│     └── ShoppingDb
│
└── SQLTEST
   ├── TestDb
   └── DemoDb
```

Each instance operates independently.

That means:

```text
SQLDEV
```

can have its own databases and configuration, while:

```text
SQLTEST
```

can have different databases and configuration.

SQL Server supports multiple Database Engine instances on one computer, and those instances run independently.

Instance vs Database

Do not confuse them.

Suppose:

```text
MYPC\SQLDEV
```

is an instance.

Inside it:

```text
EmployeeDb
ShoppingDb
CustomerDb
```

are databases.

So:

```text
Instance
  ↓
Can contain multiple databases
```

Not:

```text
Database = Instance
```

They are different concepts.

Default Instance

SQL Server can have a default instance.

The internal service instance name is:

```text
MSSQLSERVER
```

When connecting, you normally use only the computer/server name.

For example:

```text
MYPC
```

Conceptually:

```text
MYPC
↓
Default SQL Server Instance
```

A machine can have only one default SQL Server instance.

Named Instance

A named instance has a name chosen during SQL Server installation.

For example:

```text
SQLDEV
```

The connection name becomes:

```
MYPC\SQLDEV
```

Another example:

```
MYPC\SQLEXPRESS
```

SQL Server Express commonly uses a named instance such as SQLEXPRESS.

Default vs Named Instance

Default Instance	Named Instance
Only one per computer	Multiple named instances can exist
Internal instance name is MSSQLSERVER	Name chosen during installation
Connect with server/computer name	Connect with Server\Instance
Example: MYPC	Example: MYPC\SQLDEV

Easy memory:

```
Default instance:

MYPC
```

```
Named instance:

MYPC\SQLDEV
```

Multiple Instances

Suppose a computer contains:

```
MYPC
│
├── Default Instance
│
├── SQLDEV
│
└── SQLTEST
```

You could connect to:

```text
MYPC
```

for the default instance.

You could connect to:

```text
MYPC\SQLDEV
```

for the SQLDEV instance.

And:

```text
MYPC\SQLTEST
```

for the SQLTEST instance.

Only one instance can be the default; additional ones are named instances.

How to Check Your SQL Server Instance

You can execute:

```sql
SELECT @@SERVERNAME AS ServerName;
```

You can also inspect useful properties:

```sql
SELECT
  SERVERPROPERTY('ServerName') AS ServerName,
  SERVERPROPERTY('MachineName') AS MachineName,
  SERVERPROPERTY('InstanceName') AS InstanceName,
  SERVERPROPERTY('Edition') AS Edition,
  SERVERPROPERTY('ProductVersion') AS ProductVersion;
```

For a named instance, you might see something conceptually similar to:

```text
ServerName      MYPC\SQLDEV
MachineName     MYPC
InstanceName    SQLDEV
Edition         Developer Edition
ProductVersion  17.x...
```

For a default instance:

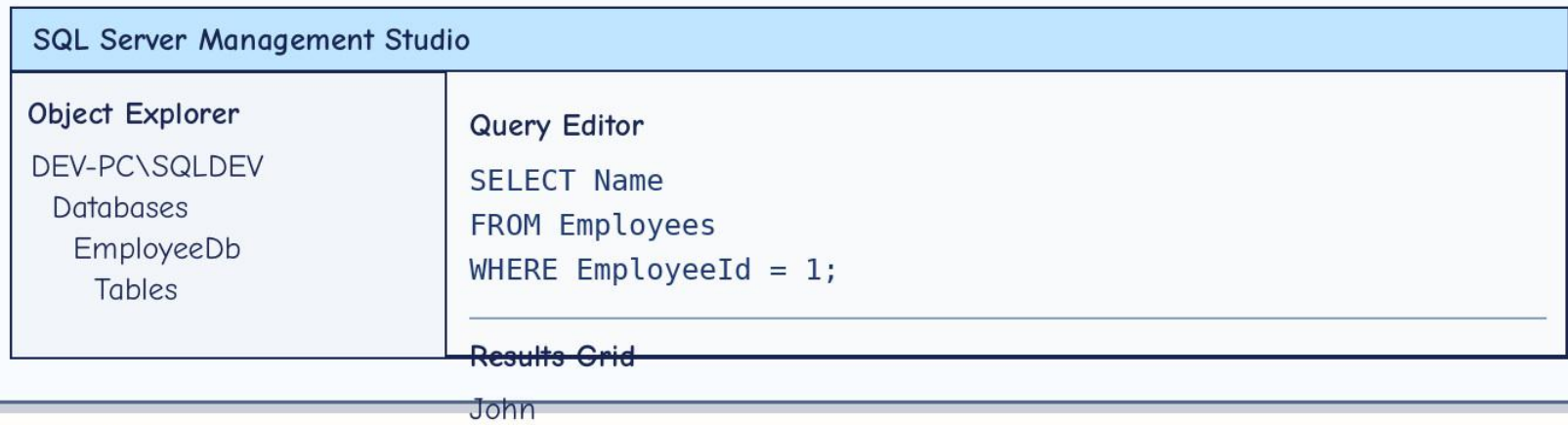
```text
InstanceName
```

can be NULL because there is no separately specified instance name.

Microsoft also recommends @@VERSION and SERVERPROPERTY as ways to identify the Database Engine version and edition.

SQL Server Management Studio — SSMS

SSMS - CLIENT TOOL VIEW



SSMS stands for:

SQL Server Management Studio

SSMS is a graphical management and development tool used to connect to and work with SQL Server and related Microsoft SQL services.

The most important distinction:

SSMS is a client tool. It is not the SQL Server Database Engine.

Think:

```text
SSMS
↓
Connects to
↓
SQL Server Instance
```

If SSMS is closed:

```text
SQL Server can still continue running.
```

Closing SSMS does not mean the database server has stopped.

Current SSMS Version

As of September 2026, SSMS 22 is the current major GA generation, and Microsoft's current release notes list SSMS 22.9.2, released August 25, 2026.

You do not need to memorize the minor version because it changes frequently.

For study notes, remember:

```text
SQL Server 2025
+
SSMS 22
```

are the current generations as of September 2026. SQL Server 2025 is Microsoft's current major SQL Server release.

Why Do We Use SSMS?

Without a graphical client, you could still connect to SQL Server using other applications or command-line tools.

SSMS makes development and administration much easier.

It provides interfaces for:

```text
Connecting to SQL Server

Viewing databases

Viewing tables

Writing SQL queries

Running queries

Viewing results

Creating databases

Creating tables

Managing database objects

Viewing server information

Managing security

Performing administrative tasks
```

Connecting Using SSMS

When SSMS opens, you connect to a SQL Server instance.

You normally provide information such as:

```text
Server / Instance
Authentication
Credentials when required
```

For a local default instance:

```text
Server name: localhost
```

or:

```text
.
```

may work depending on your installation.

For a named instance:

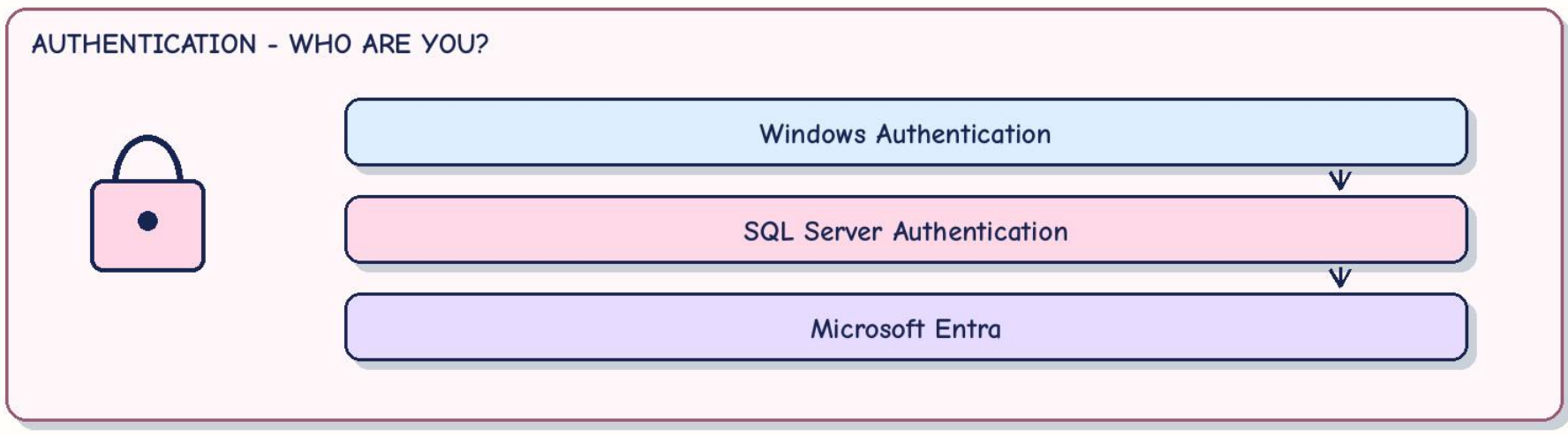
```text
.\SQLDEV
```

or:

```text
MYPC\SQLDEV
```

Microsoft documents server-name formats such as MyServer for a default instance and MyServer\MyInstance for a named instance.

Authentication in SSMS



Authentication determines who you are when connecting.

Common examples include:

```
Windows Authentication

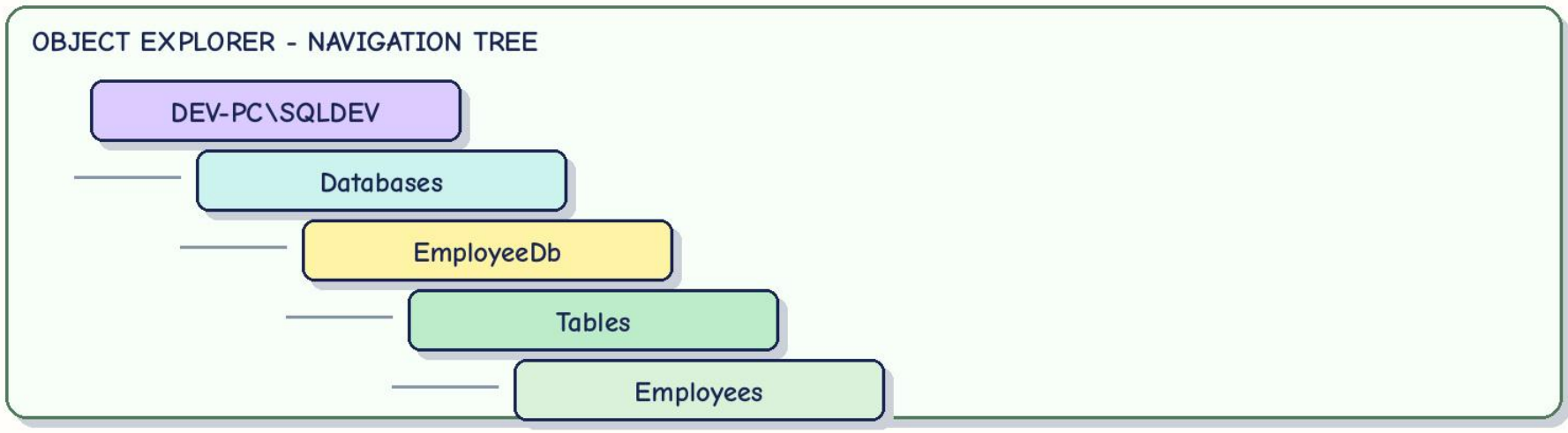
SQL Server Authentication

Microsoft Entra authentication
```

For a normal Windows-based local learning environment, Windows Authentication is often the simplest option. Microsoft recommends Windows Authentication when possible for SQL Server sign-ins.

Authentication and authorization/security will be separate topics later.

Object Explorer



After connecting, one of the most important parts of SSMS is Object Explorer.

It lets you navigate the server graphically.

You may see:

```
MYPC\SQLDEV
│
├── Databases
├── Security
├── Server Objects
├── Replication
└── Management
```

Under Databases:

```
Databases
│
├── System Databases
├── EmployeeDb
├── ShoppingDb
└── CustomerDb
```

Inside EmployeeDb:

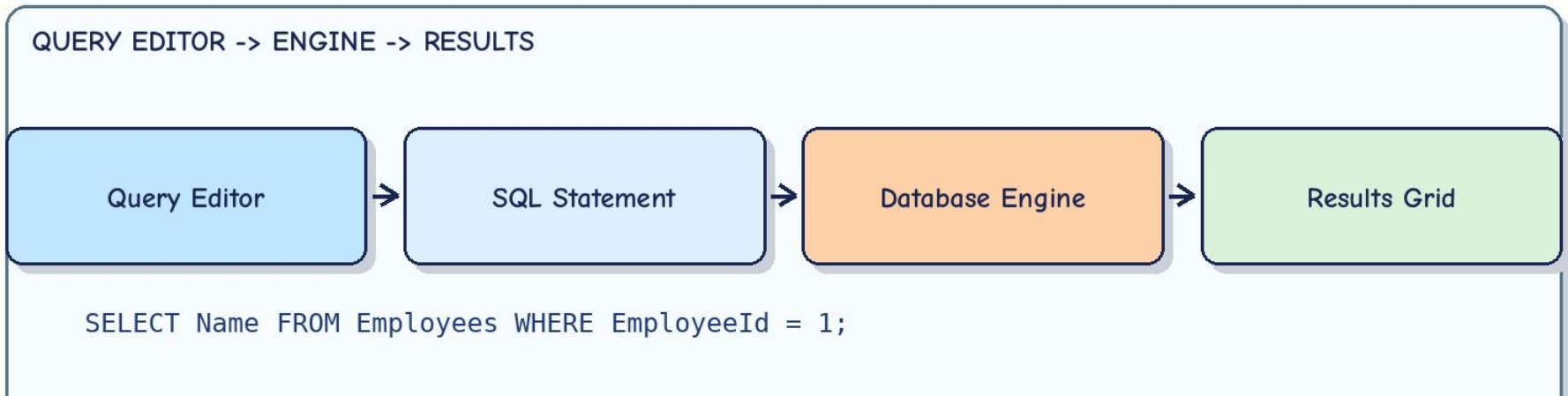
```text
EmployeeDb
│
├── Tables
├── Views
├── Programmability
├── Security
└── Storage
```

You do not need to learn every folder now.

The important idea is:

> Object Explorer lets you visually navigate the SQL Server instance and its objects.

Query Editor



The Query Editor is where you write SQL.

For example:

```sql
SELECT @@SERVERNAME;
```

or:

```sql
SELECT *
FROM Employees;
```

You can open it using:

```text
New Query
```

Then select the correct database and execute the statement.

Execute a Query

Suppose you write:




```sql
SELECT DB_NAME() AS CurrentDatabase;
```

and execute it.

The query travels:

```text
SSMS
↓
SQL Server Instance
↓
Database Engine
↓
Result
↓
SSMS Results Grid
```

The result may be:

```text
CurrentDatabase
---------------
EmployeeDb
```

Results Grid

After executing:

```sql
SELECT *
FROM Employees;
```

SSMS may display:

EmployeeId	FirstName	Department
1	John	IT
2	Priya	HR

This area is called the Results Grid.

SSMS is displaying the result.

The Database Engine actually retrieved the data.

That distinction matters:

```text
Database Engine
=
Processes query
```

TEXT (CONT.)

```
SSMS
=
Displays query and result
```

Messages Window

SSMS can also display messages.

For example:

SQL

```
PRINT 'Hello SQL Server';
```

can produce:

TEXT

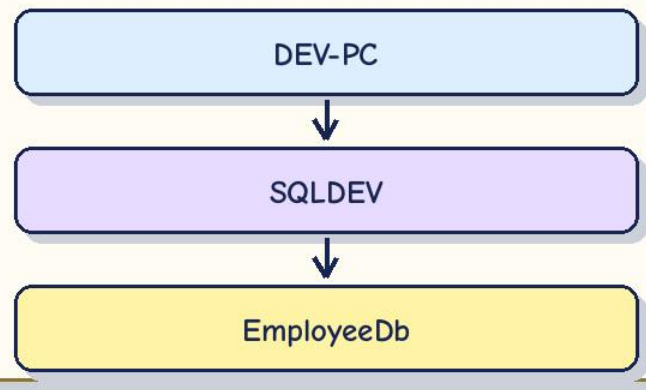
```
Hello SQL Server
```

in the Messages area.

Errors and informational messages may also appear there.

Database Selection

SERVER -> INSTANCE -> DATABASE



In the query toolbar, SSMS lets you choose the current database.

You can also explicitly write:

SQL

```
USE EmployeeDb;
GO
```

Then check:

SQL

```
SELECT DB_NAME();
```

Result:

```text
EmployeeDb
```

Always make sure you are connected to the correct:

```text
Server
Instance
Database
```

before modifying data.

SSMS Is Not SQL Server

This is a very common beginner confusion.

```text
SSMS
≠
SQL Server
```

SSMS is the tool/client.

SQL Server is the database platform/server software.

Example:

```text
SSMS
 ↓
Client tool

SQL Server Instance
 ↓
Database service
```

You can uninstall SSMS while SQL Server continues to exist.

You can also install SSMS on a different computer and connect remotely to SQL Server.

SSMS Is Not a Database

Also:

```text
SSMS
≠
Database
```

For example:



```
EmployeeDb
```

is a database.

```
SSMS
```

is a program used to work with it.

Putting Everything Together

Suppose you install SQL Server Developer Edition on:

```
DEV-PC
```

and create a named instance:

```
SQLDEV
```

The full instance name is:

```
DEV-PC\SQLDEV
```

Inside it you create:

```
EmployeeDb
```

Inside the database:

```
Employees
```

Inside the table:

```
EmployeeId | Name  | Department
---------------------------------
1          | John  | IT
2          | Priya | HR
```

You then open SSMS and connect.

The complete architecture is:



```text
SSMS
|
| Client
↓
DEV-PC\SQLDEV
|
| SQL Server Instance
↓
Database Engine
|
↓
EmployeeDb
|
| Database
↓
Employees
|
| Table
↓
Rows + Columns
```

Now you execute:

```sql
SELECT Name
FROM Employees
WHERE EmployeeId = 1;
```

What happens?

```text
1. You write SQL in SSMS.

2. SSMS sends the SQL statement to DEV-PC\SQLDEV.

3. The SQL Server instance receives the request.

4. The Database Engine processes the query.

5. It accesses EmployeeDb.

6. It finds the Employees table.

7. It finds the required row.

8. It reads the Name column.

9. SQL Server returns "John".

10. SSMS displays the result.
```

Result:

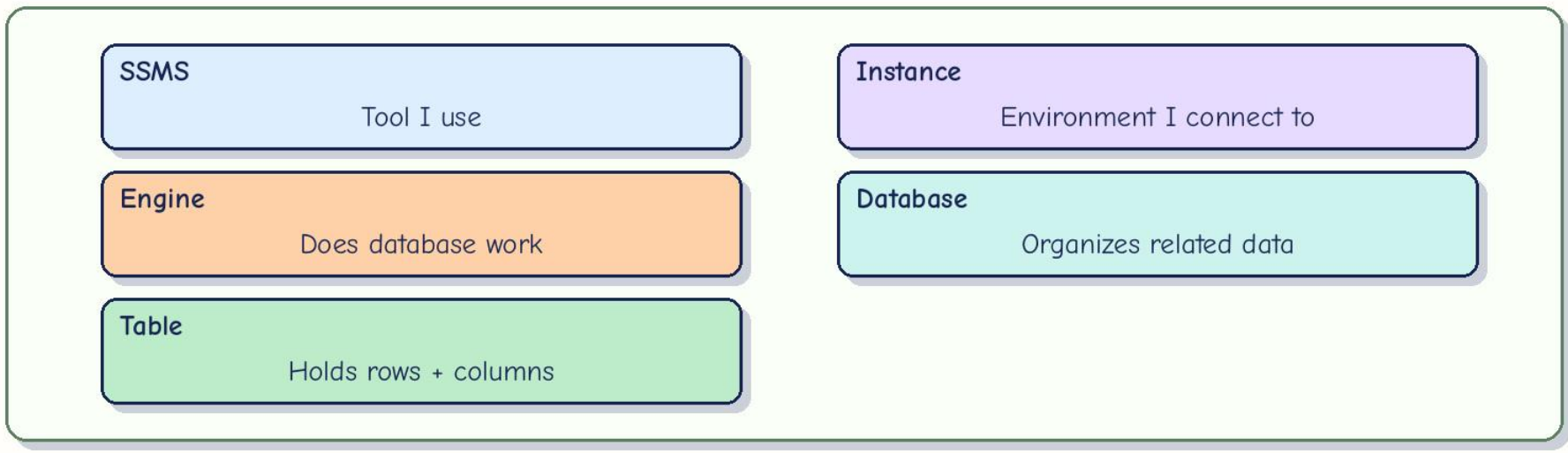
```text
Name
----
John
```

This one example connects all of the concepts in this note.

Final Comparison

Concept	What It Means	Simple Example
Row	One complete record	John's employee record
Column	One property/field	Salary
Client	Program requesting database work	SSMS
Server	System providing database service	SQL Server machine/service
Database Engine	Core service processing and managing data	Executes a SELECT
SQL Server Instance	Independent running SQL Server Database Engine environment	DEV-PC\SQLDEV
Database	Organized collection of related data	EmployeeDb
SSMS	Client management/development tool	Write and execute SQL

Easy Memory Rule



Remember this flow:

```text
SSMS
=
Tool I use

SQL Server Instance
=
Environment I connect to
```





TEXT (CONT.)

```
Database Engine
=
Component that does the database work

Database
=
Where related data is organized

Table
=
Structure containing the data

Column
=
What information is stored

Row
=
One complete record
```

Or even shorter:

TEXT

```
SSMS
↓
Instance
↓
Engine
↓
Database
↓
Table
↓
Rows + Columns
```

Interview Questions and Answers

1. What is a row in a database table?

Short Answer

A row represents one complete record in a table.

Explanation

If an Employees table contains information about employees, every employee normally occupies one row.

TEXT

```
1 | John | IT
```

represents one employee.



Interviewer Is Testing

Whether you understand the basic relational table structure.

---

2. What is a column?

Short Answer

A column represents one particular attribute or field stored for every applicable row.

Explanation

For example:

```text
EmployeeId
FirstName
Department
Salary
```

are columns of an employee table.

A column normally has a defined data type that controls the type of information it stores.

Interviewer Is Testing

Whether you understand the difference between a record and an attribute.

---

3. What is the difference between a row and a column?

Short Answer

A row represents one complete record, while a column represents one property of those records.

Example

```text
EmployeeId | Name | Department
--------------------------------
1          | John | IT
```

Name is a column.

1 | John | IT is a row.

Interviewer Is Testing

Your fundamental understanding of tables.

---

4. What is client-server architecture?

Short Answer

Client-server architecture separates the application requesting a service from the server providing that service.

Explanation

In SQL Server:

```text
Client
↓
Request
↓
SQL Server
↓
Result
↓
Client
```

SSMS or a .NET application can be a client, while SQL Server provides database services.

Interviewer Is Testing

Whether you understand how database applications communicate.

---

5. What is a database client?

Short Answer

A database client is an application or tool that connects to a database server and sends requests.

Examples

```text
SSMS
ASP.NET Core application
PowerShell application
```

Interviewer Is Testing

Whether you understand that the client is not the database itself.

---

6. Can the client and SQL Server be on the same machine?

Short Answer

Yes.

Explanation

During development, you might install both:

```text
SSMS
SQL Server
```

on your laptop.

They still have separate roles:

```text
SSMS = Client
SQL Server = Server/service
```

Interviewer Is Testing

Whether you understand client/server as logical roles rather than necessarily separate physical computers.

---

7. Can a SQL Server client be on another machine?

Short Answer

Yes.

Explanation

A .NET API on one server can connect over the network to SQL Server running on another server.

```text
API Server
  ↓
Network
  ↓
SQL Server
```

Interviewer Is Testing

Whether you understand remote database connectivity.

---

8. What is the SQL Server Database Engine?

Short Answer

The Database Engine is the core SQL Server service responsible for storing, processing, securing, and retrieving database data.

Explanation

When a client sends:

```sql
SELECT *
FROM Employees;
```

the Database Engine processes the request, accesses the required data, and returns the result.

Interviewer Is Testing

Whether you understand which SQL Server component performs core relational database operations.

---

9. What does the Database Engine do when a query is submitted?

Short Answer

It receives the request, processes the SQL statement, accesses the required database objects, performs the operation, and returns the result or status.

Example

```sql
SELECT Name
FROM Employees
WHERE EmployeeId = 1;
```

Conceptually:

```text
Receive query
  ↓
Process query
  ↓
Find required data
  ↓
Return result
```

Interviewer Is Testing

Your basic understanding of SQL Server request processing.

---

10. What is a SQL Server instance?

Short Answer

A SQL Server instance is an independent running installation/environment of the SQL Server Database Engine.

Explanation

An instance manages its own databases and operates independently from other instances on the same computer.

Example:

```text
DEV-PC\SQLDEV
```

Interviewer Is Testing

Whether you understand the difference between an installed SQL Server environment and an individual database.

---

11. Is a SQL Server instance the same as a database?

Short Answer

No.

Explanation

An instance can contain multiple databases.

```text
SQLDEV
```

TEXT (CONT.)

```
├── EmployeeDb
├── CustomerDb
└── ShoppingDb
```

SQLDEV is the instance.

The three items underneath are databases.

Interviewer Is Testing

One of the most common beginner SQL Server distinctions.

---

12. Can one computer have multiple SQL Server instances?

Short Answer

Yes.

Explanation

For example:

TEXT

```
SERVER01
│
├── SQLDEV
├── SQLTEST
└── SQLREPORTING
```

Each instance can operate independently and manage different databases.

Interviewer Is Testing

Whether you understand SQL Server instance isolation.

---

13. What is a default SQL Server instance?

Short Answer

A default instance is the single unnamed/default Database Engine instance on a computer.

Explanation

When connecting, you generally specify only:

TEXT

```
SERVER01
```

The internal default instance service name is:

TEXT

```
MSSQLSERVER
```

Only one default Database Engine instance can exist on a machine.

Interviewer Is Testing

Your knowledge of SQL Server connection naming.

14. What is a named instance?

Short Answer

A named instance is a SQL Server instance that receives an explicit name during installation.

Example

```text
SERVER01\SQLDEV
```

Here:

```text
SERVER01 = Computer/server

SQLDEV = Instance
```

Interviewer Is Testing

Whether you understand how multiple SQL Server instances are identified.

15. What is the difference between a default instance and a named instance?

Short Answer

A default instance is addressed using the server name alone, while a named instance normally requires both server and instance names.

Example

```text
Default:
SERVER01

Named:
SERVER01\SQLDEV
```

Interviewer Is Testing

Practical connection knowledge.

16. What is SSMS?

Short Answer

SSMS stands for SQL Server Management Studio. It is Microsoft's graphical client and management tool for working with SQL Server and related Microsoft SQL services.

Explanation

SSMS provides features such as:



```text
Object Explorer
Query Editor
Results
Database management
Security management
Administrative tools
```

Interviewer Is Testing

Whether you know the role of SQL Server's primary graphical management tool.

---

17. Is SSMS a database?

Short Answer

No.

Explanation

SSMS is a client application.

For example:

```text
EmployeeDb = Database

SSMS = Tool used to work with EmployeeDb
```

Interviewer Is Testing

Basic terminology.

---

18. Is SSMS the SQL Server Database Engine?

Short Answer

No.

Explanation

SSMS sends requests to the Database Engine.

```text
SSMS
↓
Database Engine
```

The Engine processes the request; SSMS provides the interface for the user.

Interviewer Is Testing

Whether you understand tooling versus server-side processing.

---

19. If I close SSMS, does SQL Server stop?

Short Answer

No.

Explanation

SSMS is only the client.

Closing:

```text
SSMS
```

does not normally stop:

```text
SQL Server Database Engine service
```

Other applications can continue connecting to SQL Server.

Interviewer Is Testing

Whether you understand the independence of client and server processes.

---

20. What is Object Explorer in SSMS?

Short Answer

Object Explorer is the hierarchical interface in SSMS used to browse SQL Server instances and database objects.

Example

```text
Server
↓
Databases
↓
EmployeeDb
↓
Tables
```

Interviewer Is Testing

Your basic familiarity with SSMS.

---

21. What is the Query Editor in SSMS?

Short Answer

The Query Editor is the area where developers write and execute SQL/T-SQL statements.

Example

```sql
SELECT *
FROM Employees;
```

SQL (CONT.)

Explanation

SSMS sends the query to the connected SQL Server instance and displays the returned result.

Interviewer Is Testing

Basic SSMS usage.

---

22. What happens when you execute a query from SSMS?

Short Answer

SSMS sends the SQL statement to the connected SQL Server instance, the Database Engine processes it, and the result is returned to SSMS.

Flow

```text
SSMS
↓
SQL statement
↓
SQL Server Instance
↓
Database Engine
↓
Database/Table
↓
Result
↓
SSMS
```

Interviewer Is Testing

Whether you understand how the major components work together rather than memorizing them independently.

---

23. How can you identify the SQL Server instance you are connected to?

Short Answer

You can inspect the connection in SSMS or execute functions such as @@SERVERNAME and SERVERPROPERTY.

Example

```sql
SELECT @@SERVERNAME AS ServerName;
```

Or:

```sql
SELECT
  SERVERPROPERTY('MachineName') AS MachineName,
  SERVERPROPERTY('InstanceName') AS InstanceName;
```

Interviewer Is Testing

Practical SQL Server troubleshooting and environment awareness.

---

24. Explain the relationship between SSMS, instance, database, table, row, and column.

Short Answer

SSMS connects to a SQL Server instance. The instance manages databases. Databases contain tables, and tables contain rows and columns.

Structure

```text
SSMS
↓
SQL Server Instance
↓
Database
↓
Table
↓
Rows + Columns
```

A row represents one record.

A column represents one attribute.

Interviewer Is Testing

Whether you have connected all the fundamental SQL Server concepts correctly.

---

25. Explain what happens when a user runs SELECT Name FROM Employees WHERE EmployeeId = 1 in SSMS.

Short Answer

SSMS sends the query to the SQL Server instance. The Database Engine processes the statement, accesses the selected database and Employees table, finds the row for employee 1, reads the Name column, and returns the value to SSMS.

Detailed Flow

```text
User
↓
SSMS
↓
SELECT Name
FROM Employees
```

TEXT (CONT.)

```
WHERE EmployeeId = 1
↓
SQL Server Instance
↓
Database Engine
↓
Database
↓
Employees Table
↓
Row where EmployeeId = 1
↓
Name column
↓
John
↓
SSMS Results Grid
```

Why This Question Is Important

This single question tests whether you understand:

TEXT

```
Client-server architecture
SSMS
SQL Server instance
Database Engine
Database
Table
Row
Column
```

instead of treating every concept as unrelated terminology.

Quick Revision

Row

TEXT

```
One complete record
```

Example:

TEXT

```
1 | John | IT
```

Column

```
TEXT
One property
```

Example:

```
TEXT
Department
```

Client

```
TEXT
Application asking SQL Server to perform work
```

Example:

```
TEXT
SSMS
```

Server

```
TEXT
System providing database services
```

Database Engine

```
TEXT
Core SQL Server service that processes
and manages relational database data
```

SQL Server Instance

```
TEXT
Independent running SQL Server environment
```

Example:

```
TEXT
DEV-PC\SQLDEV
```

Database

```
TEXT
Organized collection of related data
```

Example:

TEXT

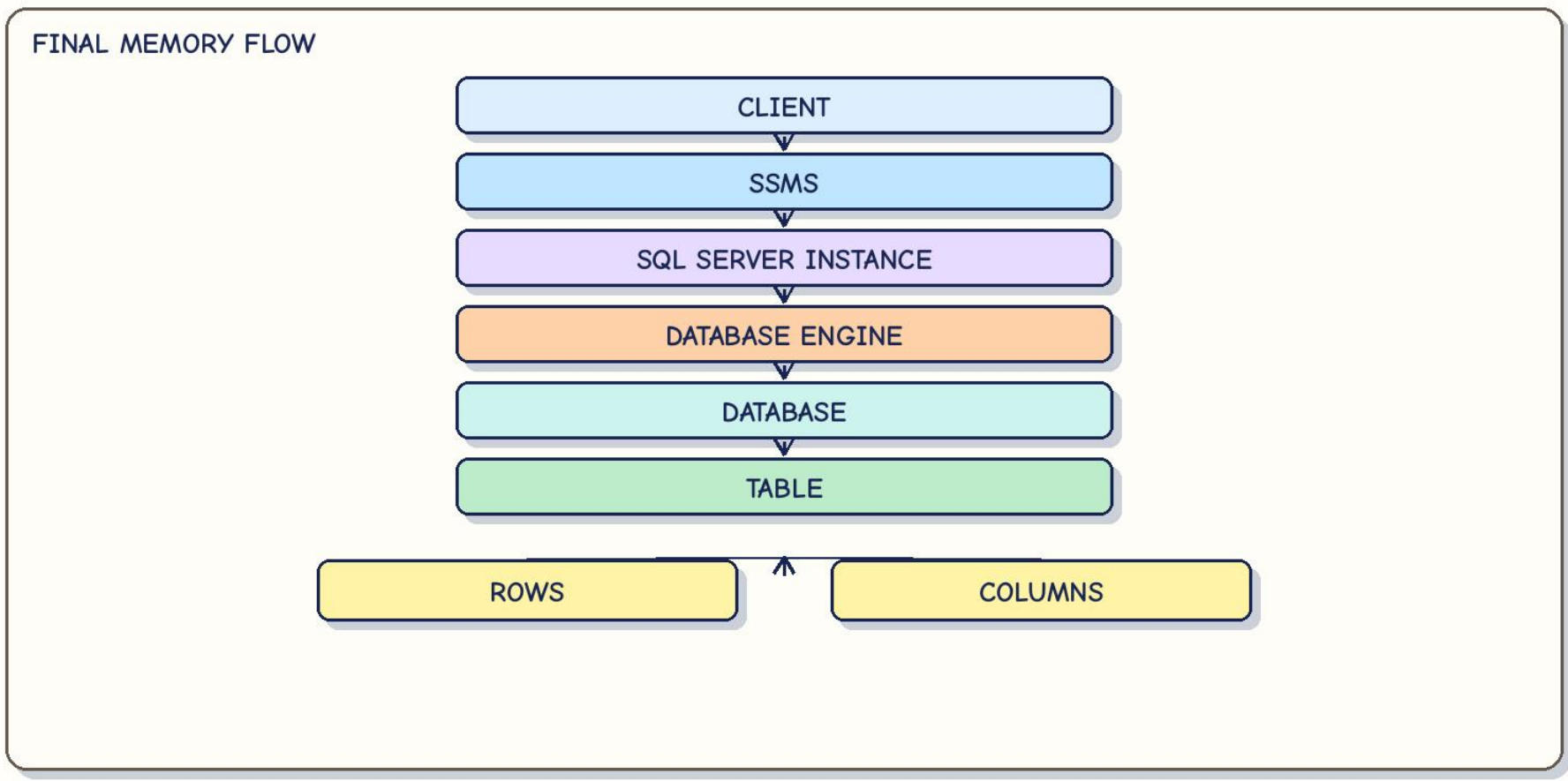
```
EmployeeDb
```

SSMS

TEXT

```
Graphical client/tool used to connect to
and manage SQL Server
```

Final Memory Diagram



The easiest sentence to remember is:

> SSMS is the client I use to connect to a SQL Server instance. The Database Engine inside that environment performs database work. The instance manages databases, databases contain tables, and tables organize data into rows and columns.